

Informe Final: Sistema de Telemetría Aeroespacial de Doble Nodo mediante ESP-NOW

Bryan Chamorro

Curso de Microcontroladores, Ingeniería Electrónica, Universidad del Cauca,
bryanchamorro@unicauca.edu.co

Fabian Narvaez

Curso de Microcontroladores, Ingeniería Electrónica, Universidad del Cauca,
leidernarvaez@unicauca.edu.co

Jhojan Perafan

Curso de Microcontroladores, Ingeniería Electrónica, Universidad del Cauca,
jhojanperafan@unicauca.edu.co

Danniel Rueda

Curso de Microcontroladores, Ingeniería Electrónica, Universidad del Cauca,
danielrueda@unicauca.edu.co

Mauricio Rodríguez Lucero

Curso de Microcontroladores, Ingeniería Electrónica, Universidad del Cauca,
erodriguezl@unicauca.edu.co

Resumen

El presente proyecto consistió en el desarrollo y validación de un sistema de telemetría de doble nodo diseñado para un cohete hidropulsado en el marco del evento Hidrochallenge. La arquitectura implementada separó las tareas de adquisición de datos y gestión de servicios web mediante dos microcontroladores ESP32 comunicados a través del protocolo de baja latencia ESP-NOW. El nodo transmisor integró sensores atmosféricos, inerciales y de posicionamiento global, consolidando la información en una estructura unificada. Por su parte, el nodo receptor operó en modo dual para capturar la telemetría y servir simultáneamente una interfaz gráfica asíncrona mediante WebSockets. Los resultados demostraron una transmisión estable a una frecuencia de 5 Hz, permitiendo el monitoreo dinámico de variables críticas y coordenadas geográficas en tiempo real desde una estación terrena independiente.

ÍNDICE

1. REQUISITOS	3
Arquitectura del Sistema y Entorno de Desarrollo	3
Flujo Unificado de Telemetría	3
2. DIAGRAMA CIRCUITAL	5
Validación en Simulación Digital	5
Implementación Física del Nodo de Sensado	5
3. DIAGRAMA DE FLUJO DEL PROGRAMA	6
Lógica del Transmisor y Validación de Consola	6
Arquitectura Web y Dashboard en Tiempo Real	8
Backend y Gestión de Comunicación Bidireccional	8
Frontend y Renderizado Dinámico de Telemetría	9
4. CONCLUSIONES Y LECCIONES APRENDIDAS	11
REFERENCIAS	12

1 REQUISITOS

Arquitectura del Sistema y Entorno de Desarrollo

El sistema se fundamenta en un esquema de transmisión inalámbrica punto a punto para evitar retardos en la lectura de sensores. El desarrollo se realizó en el entorno PlatformIO, integrando librerías especializadas para la gestión de protocolos I2C, UART y servicios web asíncronos. La Figura 1 detalla el inventario de librerías validadas para garantizar la concurrencia del firmware. PlatformIO. (2024). *PlatformIO Documentation*. Recuperado de <https://docs.platformio.org/>

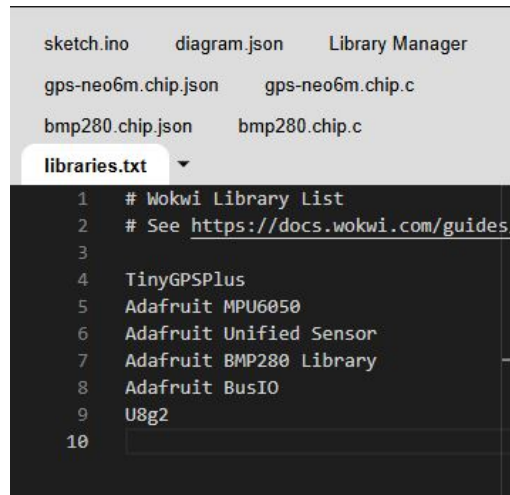


Figura 1. Librerías técnicas empleadas para el GPS, IMU y servidor web. Fuente: elaboración propia.

Flujo Unificado de Telemetría

La integración de los subsistemas (BMP280, MPU6050 y GPS) se consolidó en una estructura de datos denominada *struct_message*. Este paquete es difundido mediante ESP-NOW hacia el receptor, el cual procesa la información para su publicación web. La Figura 2 ilustra la topología completa de los componentes. Bosch Sensortec. (2021). *BMP280 Digital Pressure Sensor Datasheet*. Bosch Sensortec GmbH.

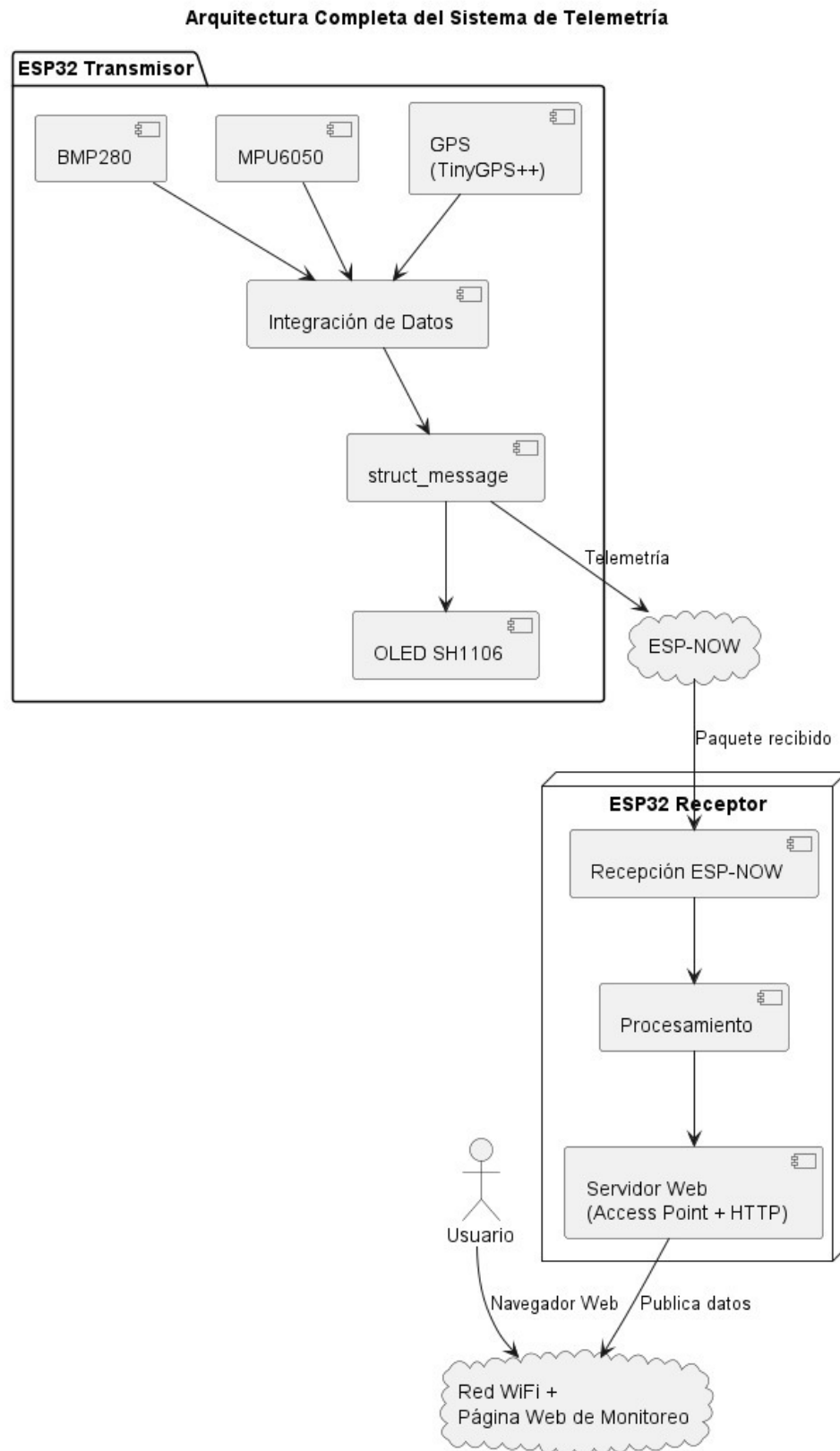


Figura 2. Arquitectura completa del flujo de datos desde el sensado hasta la visualización final.
Fuente: elaboración propia.

2 DIAGRAMA CIRCUITAL

Validación en Simulación Digital

Antes del montaje físico, se utilizó la plataforma Wokwi para certificar el conexionado de los periféricos. Se validó el bus I2C para el sensor barométrico y la pantalla OLED, asegurando que no existieran conflictos de direccionamiento. Wokwi. (2024). *Wokwi Online Electronics Simulator*. Recuperado de <https://wokwi.com/>

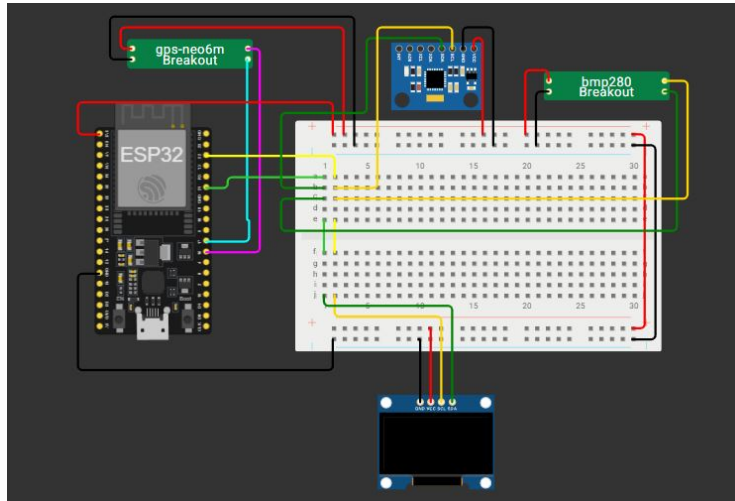


Figura 3. Diagrama de simulación del hardware del nodo transmisor. Fuente: elaboración propia.

Implementación Física del Nodo de Sensado

El montaje real se distribuyó de manera que el sensor BMP280 quedara aislado del calor residual del procesador. La Figura 4 presenta el nodo transmisor integrado, listo para la fase de adquisición de datos en rampa de lanzamiento .

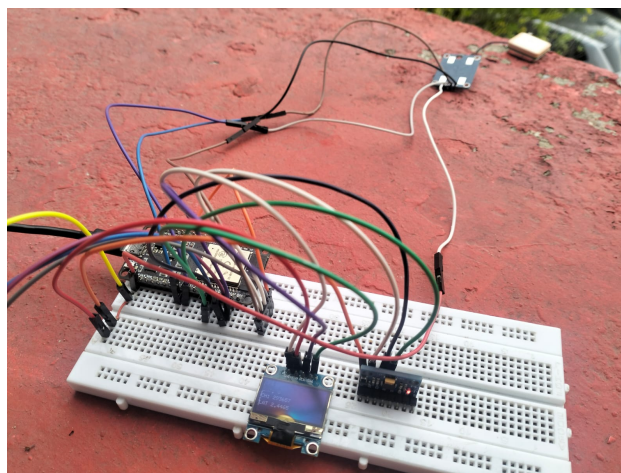


Figura 4. Implementación física de los sensores atmosféricos e inerciales en protoboard. Fuente: elaboración propia.

3 DIAGRAMA DE FLUJO DEL PROGRAMA

Lógica del Transmisor y Validación de Consola

El firmware del transmisor utiliza una ejecución no bloqueante para leer el GPS asincrónicamente y enviar paquetes cada 500 ms. Las Figuras 5, 6 y 7 evidencian la correcta inicialización del sistema y la captura de datos reales, incluyendo coordenadas de latitud 2,44 y longitud -76,61 obtenidas en exteriores. U-blox. (2011). *NEO-6 u-blox 6 GPS Modules Data Sheet*. u-blox AG.

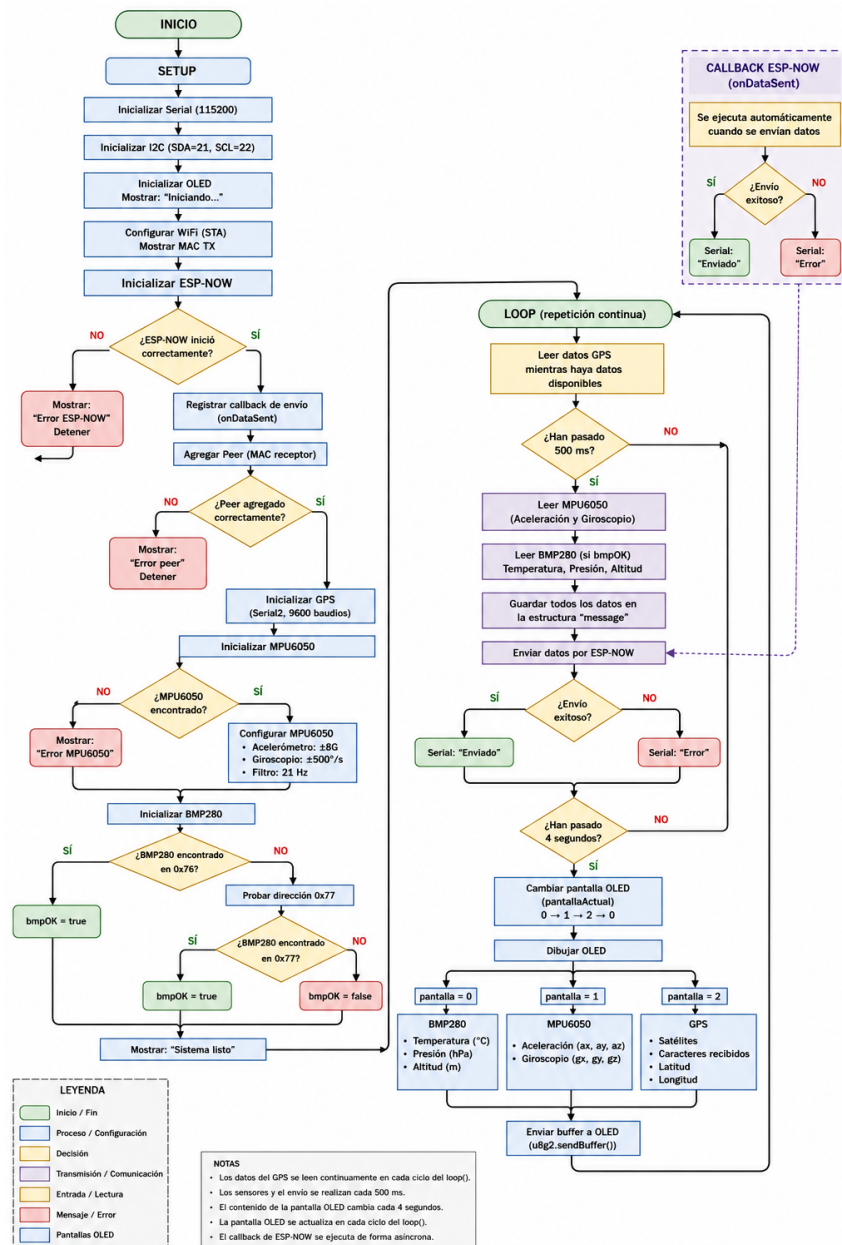


Figura 5. Diagrama de flujo detallado de la lógica algorítmica del nodo transmisor. Fuente: elaboración propia.

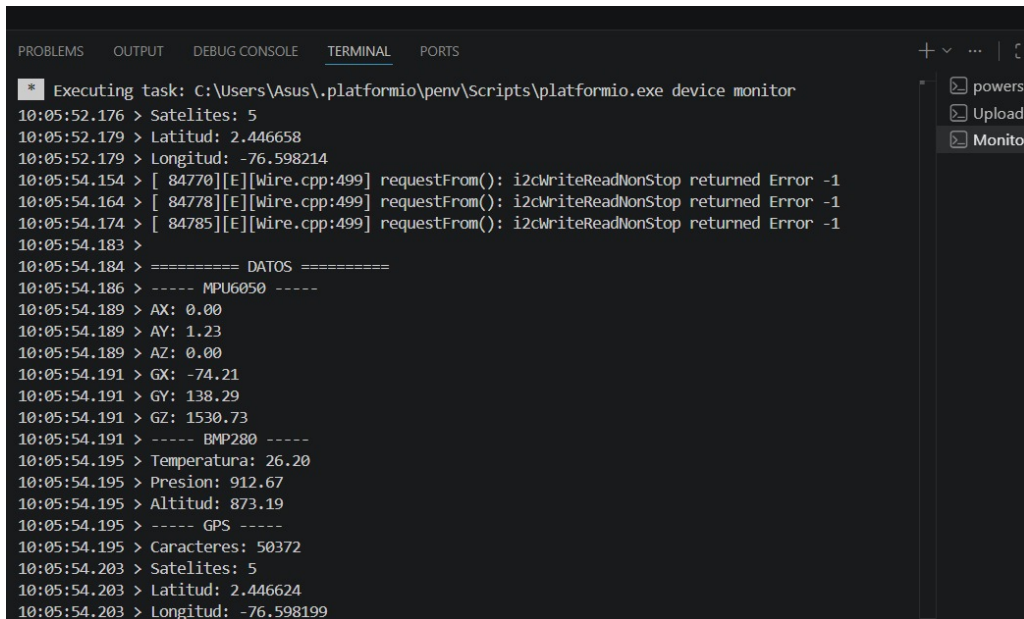
```
SERIAL MONITOR  CHIPS CONSOLE
=====
INICIANDO SISTEMA
=====
OLED SSD1306 iniciada
GPS iniciado en UART2
GPS TX -> GPIO 16
GPS RX -> GPIO 17
MPU6050 encontrado en 0x68
BMP280 encontrado en 0x76
=====

----- BMP280 -----
Temperatura: 25.08 C
Temperatura calibrada: 21.08 C
Presion: 1006.53 hPa
Altitud: 56.08 m
----- MPU6050 -----
```

Figura 6. Monitor serial confirmando la inicialización de buses I2C y detección de sensores. Fuente: elaboración propia.

```
SERIAL MONITOR  CHIPS CONSOLE
=====
Temperatura calibrada: 21.08 C
Presion: 1006.53 hPa
Altitud: 56.08 m
----- MPU6050 -----
AX: 0.00 m/s2
AY: 0.00 m/s2
AZ: -9.81 m/s2
GX: 0.00 grados/s
GY: 0.00 grados/s
GZ: 0.00 grados/s
----- GPS -----
Caracteres recibidos: 280
Latitud: 2.444800
Longitud: -76.614700
Satelites: 8
Altitud GPS: 1760.00 m
```

Figura 7. Detalle de la adquisición dinámica de variables inerciales y atmosféricas. Fuente: elaboración propia.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
* Executing task: C:\Users\Asus\.platformio\penv\Scripts\platformio.exe device monitor
10:05:52.176 > Satelites: 5
10:05:52.179 > Latitud: 2.446658
10:05:52.179 > Longitud: -76.598214
10:05:54.154 > [ 84770][E][Wire.cpp:499] requestFrom(): i2cwriteReadNonStop returned Error -1
10:05:54.164 > [ 84778][E][Wire.cpp:499] requestFrom(): i2cwriteReadNonStop returned Error -1
10:05:54.174 > [ 84785][E][Wire.cpp:499] requestFrom(): i2cwriteReadNonStop returned Error -1
10:05:54.183 >
10:05:54.184 > ===== DATOS =====
10:05:54.186 > ----- MPU6050 -----
10:05:54.189 > AX: 0.00
10:05:54.189 > AY: 1.23
10:05:54.189 > AZ: 0.00
10:05:54.191 > GX: -74.21
10:05:54.191 > GY: 138.29
10:05:54.191 > GZ: 1530.73
10:05:54.191 > ----- BMP280 -----
10:05:54.195 > Temperatura: 26.20
10:05:54.195 > Presion: 912.67
10:05:54.195 > Altitud: 873.19
10:05:54.195 > ----- GPS -----
10:05:54.195 > Caracteres: 50372
10:05:54.203 > Satelites: 5
10:05:54.203 > Latitud: 2.446624
10:05:54.203 > Longitud: -76.598199
```

Figura 8. Recepción de telemetría consolidada y posicionamiento GPS activo. Fuente: elaboración propia.

Arquitectura Web y Dashboard en Tiempo Real

La culminación técnica del proyecto reside en la implementación de una interfaz de monitoreo de alta fidelidad, la cual separa la capa de adquisición de datos de la capa de visualización. Esta arquitectura se fundamenta en un servidor web embebido dentro del ESP32 receptor, configurado en modo dual (SoftAP + STA) para permitir la escucha de paquetes ESP-NOW mientras sostiene el punto de acceso inalámbrico.

Backend y Gestión de Comunicación Bidireccional

El núcleo del servidor se desarrolló utilizando las librerías *ESPAsyncWebServer* y *AsyncTCP*, las cuales operan bajo un modelo orientado a eventos que no bloquea la ejecución del procesador. A diferencia de los servidores HTTP tradicionales, se implementó el protocolo **WebSockets** para establecer un canal de comunicación dúplex completo entre el microcontrolador y los clientes conectados.

Durante la operación, el receptor captura la estructura de datos proveniente del transmisor, la transforma en una cadena serializada mediante la librería *ArduinoJson* y la difunde masivamente a través del túnel de WebSockets a una tasa de refresco de 5 Hz (cada 200 ms). Para evitar desbordamientos de memoria RAM, el firmware ejecuta de forma recurrente la rutina *ws.cleanupClients()*, garantizando la estabilidad del sistema durante sesiones prolongadas de telemetría. Me-no-dev. (2024). *ESPAsyncWebServer and AsyncTCP Libraries*. GitHub Repository. Recuperado de <https://github.com/me-no-dev/ESPAsyncWebServer>

Diagrama de interacción entre ESP32 y navegador

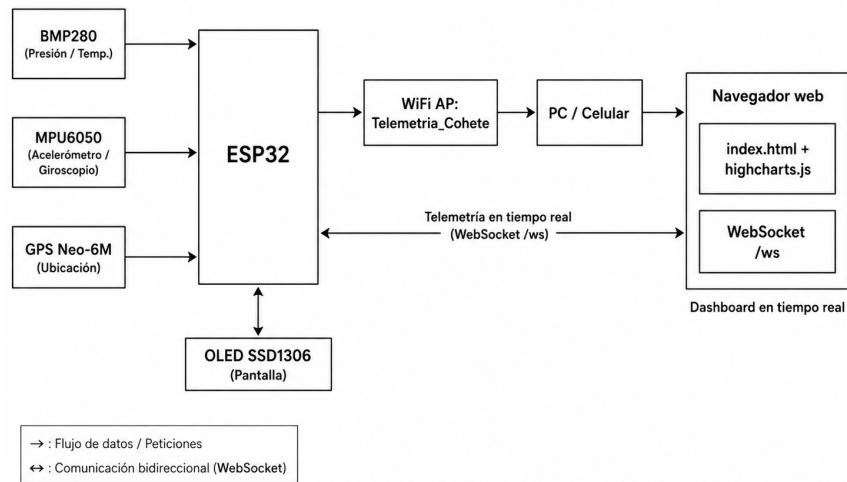


Figura 9. Interacción arquitectónica entre el servidor ESP32, el sistema de archivos SPIFFS y el navegador web. Fuente: elaboración propia.

Frontend y Renderizado Dinámico de Telemetría

La interfaz de usuario (Dashboard) se aloja en la memoria flash interna del ESP32 mediante el sistema de archivos SPIFFS. El *frontend* fue diseñado con HTML5 y CSS3 para ofrecer una visualización nítida de variables críticas como aceleración triaxial, altitud barométrica, temperatura y coordenadas geográficas.

La lógica de visualización en el cliente web se basa en un script de JavaScript que procesa los paquetes JSON entrantes sin recargar la página. Se integró la librería **Highcharts.js** para generar series de tiempo en tiempo real. Cada vez que el cliente recibe un mensaje WebSocket, el script parsea el objeto de datos y añade un nuevo punto a los vectores de las gráficas, manteniendo un historial deslizable de los últimos 20 puntos de datos para facilitar el análisis de tendencias durante el vuelo.

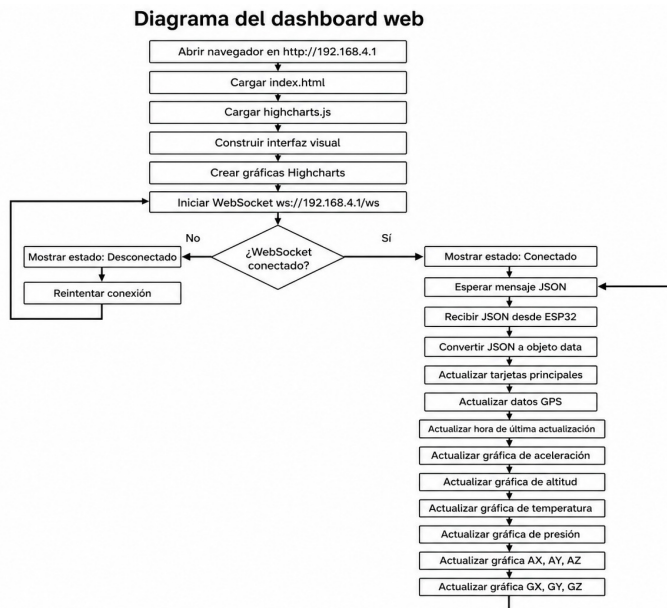


Figura 10. Diagrama de flujo del procesamiento de telemetría y actualización asíncrona del Dashboard. Fuente: elaboración propia.

Como se evidencia en las pruebas finales de integración, el sistema permite un monitoreo dinámico donde las variaciones inerciales registradas por el MPU6050 y los cambios de altitud detectados por el BMP280 se reflejan instantáneamente en la estación terrena, incluso en condiciones de desconexión satelital (GPS Sin Fix), asegurando la integridad de la misión.

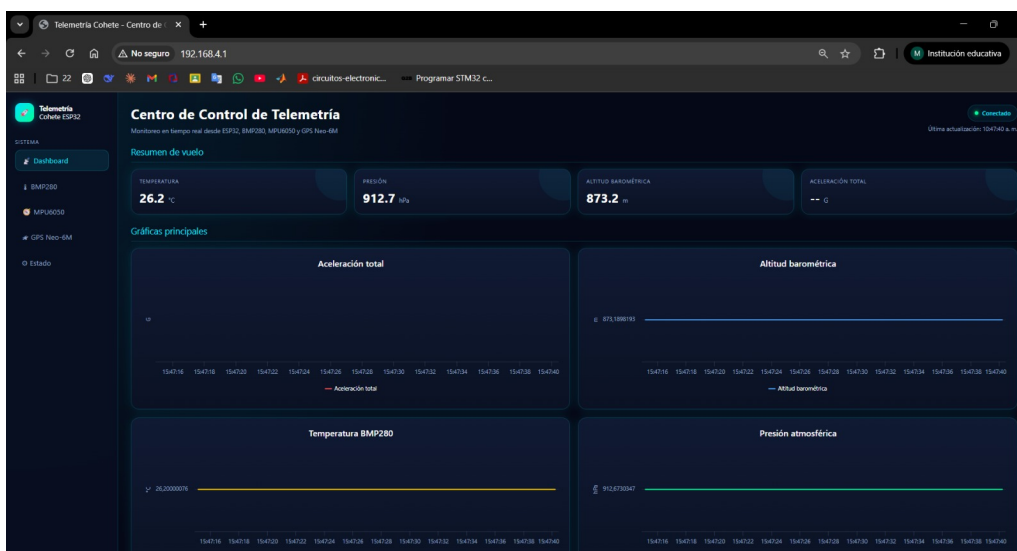


Figura 11. Interfaz operativa del Dashboard visualizando el resumen de vuelo y gráficas dinámicas de los sensores. Fuente: elaboración propia.

4 CONCLUSIONES Y LECCIONES APRENDIDAS

A partir del diseño, simulación e implementación física del sistema de telemetría de doble nodo para el proyecto Hydrochallenge, se derivan conclusiones fundamentales que validan la arquitectura técnica seleccionada. En primera instancia, se demostró que la separación de tareas mediante dos microcontroladores ESP32 es la solución óptima para misiones aeroespaciales de alta exigencia. Mientras el nodo transmisor se dedica exclusivamente a la adquisición de datos a alta frecuencia, el nodo receptor gestiona la carga computacional pesada que implica mantener un servidor web asíncrono y la difusión de datos mediante WebSockets. Esta arquitectura evitó cuellos de botella en el procesamiento y garantizó una tasa de refresco constante de 5 Hz en la estación terrena, algo inalcanzable con un único procesador que deba gestionar simultáneamente sensores y red.

En el ámbito de las telecomunicaciones, la validación del protocolo ESP-NOW frente a plataformas IoT comerciales como Adafruit IO arrojó resultados determinantes. Se concluyó que la dependencia de infraestructuras externas y nubes públicas es inviable para lanzamientos en rampa debido a las latencias impredecibles y las restricciones de transferencia de datos (*rate limits*). La creación de una red local independiente (SoftAP) permitió una comunicación robusta y de baja latencia, asegurando que la telemetría del cohete sea accesible incluso en zonas remotas sin cobertura de datos móviles.

Respecto a la instrumentación inercial y atmosférica, el proyecto permitió identificar que los sensores comerciales requieren calibraciones críticas por software antes de ser considerados aptos para la misión. Se determinó que el barómetro BMP280 sufre de una deriva térmica significativa provocada por el calor residual del microcontrolador y la pantalla OLED en montajes compactos. La implementación de un offset algorítmico de -4 °C fue necesaria para normalizar las lecturas y obtener una estimación de altitud barométrica coherente con el entorno real. De igual manera, la reasignación de ejes en el acelerómetro MPU6050 subrayó la importancia de alinear el sistema de coordenadas digital con la vertical física del cohete para que las aceleraciones registradas durante el despegue sean interpretadas correctamente por la estación terrena.

La integración del módulo GPS NEO-6M proporcionó una lección logística vital sobre el tiempo de preparación en rampa. Las pruebas ratificaron que el dispositivo requiere una línea de visión directa a la constelación satelital y un tiempo de estabilización (*Time-To-First-Fix*) que puede superar los tres minutos en condiciones de arranque en frío. Esto establece que el sistema de telemetría debe ser energizado con suficiente antelación al lanzamiento para asegurar que la transmisión incluya coordenadas válidas desde el momento del despegue.

Finalmente, a nivel de hardware y manufactura, el paso de la simulación en Wokwi al montaje físico en protoboard reveló que la mayor fuente de errores en la comunicación I2C y UART son los falsos contactos y el ruido electromagnético inducido por cables tipo jumper. Esta experiencia justifica de manera técnica la transición obligatoria hacia circuitos impresos (PCB) diseñados a medida. El uso de PCBs soldadas no solo reducirá el ruido en las señales de los sensores, sino que garantizará la integridad estructural del sistema electrónico frente a las vibraciones y altas aceleraciones de impacto que experimentará el cohete hidropulsado durante su operación.

REFERENCIAS

Adafruit Industries. (2024). *Adafruit BMP280 Barometric Pressure + Temperature Sensor Breakout Guide*. Adafruit Learning System.

Adafruit Industries. (2024). *Monochrome OLED Breakouts – SSD1306*. Adafruit Learning System.

Arduino. (2024). *Wire Library Documentation*. Arduino Documentation. Recuperado de <https://www.arduino.cc/reference/en/language/functions/communication/wire/>

Banzi, M., & Shiloh, M. (2022). *Getting Started with Arduino* (4ta ed.). Sebastopol, CA, USA: O'Reilly Media.

Blanchon, B. (2024). *ArduinoJson Library*. GitHub Repository. Recuperado de <https://github.com/bblanchon/ArduinoJson>

Bosch Sensortec. (2021). *BMP280 Digital Pressure Sensor Datasheet*. Bosch Sensortec GmbH.

Espressif Systems. (2023). *ESP32 Series Datasheet*. Espressif Systems.

Hart, M. (2024). *TinyGPSPlus Library*. GitHub Repository. Recuperado de <https://github.com/mikalhart/TinyGPSPlus>

Highsoft AS. (2024). *Highcharts JS*. Recuperado de <https://www.highcharts.com/>

InvenSense. (2013). *MPU-6000 and MPU-6050 Product Specification Revision 3.4*. InvenSense Inc.

Kolban, N. (2017). *Kolban's Book on ESP32* (1ra ed.). Texas, USA: Leanpub.

Me-no-dev. (2024). *ESPAsyncWebServer and AsyncTCP Libraries*. GitHub Repository. Recuperado de <https://github.com/me-no-dev/ESPAsyncWebServer>

Olikraus. (2024). *U8g2 Library for Monochrome Displays*. GitHub Repository. Recuperado de <https://github.com/olikraus/u8g2>

PlatformIO. (2024). *PlatformIO Documentation*. PlatformIO. Recuperado de <https://docs.platformio.org/>

u-blox. (2011). *NEO-6 u-blox 6 GPS Modules Data Sheet*. u-blox AG.

Wokwi. (2024). *Wokwi Online Electronics Simulator*. Recuperado de <https://wokwi.com/>